

SMART SERVICE BOOKING SYSTEM

Project Handbook

A Django-based platform connecting customers with local service providers

Version 1.0 · Prepared January 2026

Built with Django 6.0 · Bootstrap 5 · SQLite

Table of Contents

1. Overview
2. Technology Stack
3. Project Structure
4. User Roles & Accounts
5. Feature Reference
6. Data Model
7. URL Reference
8. Local Setup & Installation
9. Deployment
10. Common Operating Procedures
11. Known Limitations & Roadmap

1. Overview

Smart Service Booking System is a Django web application that lets local service providers (salons, tutors, cleaners, therapists, and similar businesses) list their services and availability, and lets customers search, browse, and book appointments online. The system supports three account types — Customer, Service Provider, and Superadmin — each with a dedicated dashboard.

This handbook documents the application's architecture, features, data model, and day-to-day operating procedures, and is meant as the primary reference for anyone developing, deploying, or maintaining the project.

1.1 Purpose of this Document

- Give new contributors a working mental model of the codebase.
- Document every user-facing feature and the URLs that expose it.
- Describe the database schema and how the core objects relate.
- Capture setup, deployment, and maintenance procedures in one place.

1.2 At a Glance

Attribute	Detail
Framework	Django 6.0.2 (Python)
Front end	Bootstrap 5.3, Bootstrap Icons, vanilla JS
Database	SQLite (db.sqlite3)
Static files	WhiteNoise middleware
Deployment target	Render (gunicorn + Procfile)
Apps	accounts (identity & dashboards), bookings (services, availability, bookings)
Account types	Customer, Service Provider, Superadmin

2. Technology Stack

2.1 Core Dependencies

The dependencies declared in requirements.txt:

Package	Role
Django 6.0.2	Web framework — models, views, templates, auth, admin
gunicorn	Production WSGI server used on Render
whitenoise	Serves collected static files directly from Django in production

Package	Role
openai / pydantic / httpx	Available in the environment for future AI-assisted features
sqlparse	Django's SQL formatting dependency

2.2 Front End

- Bootstrap 5.3 and Bootstrap Icons, loaded via CDN in the base templates.
- Google Fonts (Poppins for auth pages, Inter for dashboards) for typography.
- Two global stylesheets: static/css/base.css for public pages and static/css/dashboard.css for authenticated pages, plus page-specific CSS files (browse.css, home.css, register.css, success.css, superadmin.css).
- static/js/main.js provides shared behaviour: mobile menu toggle, active-link highlighting, form validation helpers, toast notifications, and auto-dismissing alerts.

3. Project Structure

The project follows Django's standard layout with two apps and a shared, global static/templates directory (rather than per-app templates), so styling and markup stay consistent across the site.

```
smart-service-booking-system/
├── manage.py
├── requirements.txt
├── Procfile
├── db.sqlite3
├── booking_system/
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py / asgi.py
├── accounts/
│   ├── models.py
│   ├── views.py
│   ├── urls.py
│   └── management/commands/
├── bookings/
│   ├── models.py
│   ├── views.py
│   ├── signals.py
│   └── urls.py
├── templates/
│   ├── accounts/
│   ├── bookings/
│   └── superadmin/
├── static/
│   ├── css/
│   └── js/
```

gunicorn start command for Render
SQLite database
Project settings
Root URLconf (includes accounts + bookings)
Identity, auth, dashboards, superadmin
UserProfile
Services, availability, bookings
ProviderProfile, Service, Availability, Booking, ...
Global templates (base.html, dashboard_base.html, ...)
Global CSS/JS

3.1 Template Inheritance

Two base templates anchor the UI:

- `base.html` — used by public pages (landing page, register, login). Provides Bootstrap, fonts, and the blocks: `title`, `extra_css`, `header_subtitle`, `content`, `extra_js`.
- `dashboard_base.html` — used by every authenticated page (customer dashboard, provider tools, superadmin panel). Provides the sidebar navigation and the blocks: `title`, `extra_css`, `page_title`, `page_subtitle`, `header_actions`, `dashboard_content`, `extra_js`.

New pages should extend one of these two templates rather than duplicating markup, per the pattern documented in `PROJECT_STRUCTURE.md`.

4. User Roles & Accounts

Every account is a standard Django User plus a `UserProfile` (accounts app) that records a `user_type`. Providers additionally get a `ProviderProfile` (bookings app) holding their business details.

4.1 Customer

- Registers via `/register/user/` — collects name, contact details, birthday, and address.
- Browses and searches providers/services, views availability, and books appointments.
- Manages bookings from 'My Bookings' and receives notifications.

4.2 Service Provider

- Registers via `/register/provider/` — collects business name, KVK (Chamber of Commerce) number, city, bio, and service type.
- Creates and manages Services (name, category, price, duration).
- Publishes Availability slots per service and manages incoming bookings.

4.3 Superadmin

- Identified by `user_type = 'superadmin'` on the `UserProfile`.
- Created with the management command: `python manage.py create_superuser` (default credentials `username superadmin / password admin123` — must be changed after first login).
- Every superadmin view is protected by the `superadmin_required` decorator; unauthorized visitors are redirected to the regular dashboard.
- Gets a dedicated control panel covering platform analytics, user management, service management, and bulk notifications.

5. Feature Reference

5.1 Registration, Login & Dashboard

- Landing page with two registration paths (Customer vs Provider).
- Django's `UserCreationForm`-based registration, auto-login on success.
- Custom login/logout views with a glassmorphism-styled login page.

- Role-aware dashboard showing account status, member-since date, last login, and unread notifications.

5.2 Service Management (Provider)

- Providers create, edit, delete, and toggle the active status of services from 'My Services'.
- Each service has a category, price (EUR), and duration (30 minutes to 4 hours).

5.3 Availability

- Providers publish availability slots (date, start/end time) per service.
- Customers view a provider's open slots before booking; a booked slot links 1-to-1 to its Booking record so it can't be double-booked.

5.4 Search & Browse

- Customers can browse all providers or run a filtered search (category, location, price range) via /search/.
- Every search is logged in SearchQuery for analytics — query text, filters used, result count, and the service eventually clicked, if any.

5.5 Booking Flow

- Customer selects a service and an open slot, then confirms the booking (confirm_booking).
- A Booking record is created with status pending, and moves through confirmed / completed / cancelled.
- Providers manage incoming requests from 'Provider Bookings'; customers manage theirs from 'My Bookings' and can cancel (cancel_booking).
- A calendar view (booking_calendar) gives a visual schedule of upcoming bookings.

5.6 Notifications

- In-app notifications (booking, cancellation, reminder, system, message) with an is_read flag and optional deep link.
- Generated automatically via Django signals (bookings/signals.py) on key events such as new bookings.

5.7 Superadmin Control Panel

- Dashboard analytics — totals for users, services, bookings, and revenue; breakdown by role; top services by booking count.
- User management — searchable, filterable list of all accounts with join date and last login.
- Service management — platform-wide service list, filterable by active/inactive status.
- Bulk notifications — send a message to all users, customers only, or providers only.

6. Data Model

The schema spans two apps. `accounts.UserProfile` extends Django's built-in `User` with a role flag; `bookings.py` holds everything related to the marketplace itself.

6.1 `accounts.UserProfile`

Field	Notes
<code>user</code>	OneToOneField → User
<code>user_type</code>	user / provider / superadmin
<code>phone_number</code>	Shared by all roles
<code>birthday</code> , <code>address</code>	Customer-specific
<code>city</code> , <code>bio</code> , <code>service_type</code> , <code>kvk_number</code>	Provider-specific (legacy — see <code>ProviderProfile</code> below)

6.2 `bookings.ProviderProfile`

The current, dedicated provider table (superseded the fields on `UserProfile`). Holds `business_name`, `kvk_number`, `service_type`, `bio`, `city`, `address`, `phone_number`, `website`, `years_experience`, `rating`, `total_bookings`, `is_verified`, and `is_active`, plus `created_at` / `updated_at` timestamps.

6.3 `bookings.Service`

A service offered by a provider: `name`, `category`, `description`, `price` (EUR), `duration` (30–240 minutes), and `is_active`. Linked to its provider via a `ForeignKey`.

6.4 `bookings.Availability`

A single bookable time window: `provider`, an optional linked `service`, `date`, `start_time`, `end_time`, and `is_available`.

6.5 `bookings.Booking`

Field	Notes
<code>customer</code> / <code>provider</code>	FKs to User
<code>service</code>	FK to Service
<code>availability</code>	OneToOne → Availability (prevents double-booking)
<code>date</code> , <code>start_time</code> , <code>end_time</code>	Copied from the slot at booking time
<code>price</code>	Snapshot of the service price when booked
<code>status</code>	pending / confirmed / completed / cancelled

Field	Notes
customer_notes, provider_notes	Free text

6.6 bookings.Notification

Belongs to a user; has notification_type, title, message, is_read, and an optional link for click-through.

6.7 bookings.SearchQuery

Analytics record of a search: query text, filters (category, location, min/max price), results_count, the service clicked (if any), and requester IP.

6.8 Deprecated: bookings.OldProvider

Kept only for historical data (4 legacy rows) and mapped to the old bookings_provider table. **Do not use this model for new code — use ProviderProfile instead.**

7. URL Reference

7.1 accounts app

Path	View	Purpose
/	home	Landing page
/get-started/	index	Account-type selection
/register/user/	register_user	Customer sign-up
/register/provider/	register_provider	Provider sign-up
/login/ · /logout/	custom_login / custom_logout	Authentication
/dashboard/	dashboard	Role-aware dashboard (login required)
/calendar/	booking_calendar	Booking calendar view
/profile/ · /notifications/	profile / notifications	Account settings & alerts
/superadmin/*	superadmin_dashboard / users / services / notifications	Superadmin panel

7.2 bookings app

Path	View	Purpose
/browse-providers/	browse_providers	Browse all providers
/search/	search_services	Filtered search

Path	View	Purpose
/service/<id>/availability/	view_availability	View a service's open slots
/confirm/<id>/	confirm_booking	Confirm a booking
/my-bookings/	my_bookings	Customer's bookings
/cancel-booking/<id>/	cancel_booking	Cancel a booking
/my-services/ · /services/add edit delete toggle/	my_services etc.	Provider service CRUD
/add-availability/ · /delete-availability/<id>/	add_availability etc.	Provider availability CRUD
/provider/bookings/	provider_bookings	Provider's incoming bookings

8. Local Setup & Installation

8.1 Prerequisites

- Python 3.x
- pip and a virtual environment tool (venv)

8.2 Steps

```
# 1. Clone/copy the project, then move into it
cd smart-service-booking-system

# 2. Create and activate a virtual environment
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate

# 3. Install dependencies
pip install -r requirements.txt

# 4. Apply database migrations
python manage.py migrate

# 5. Create an admin/superadmin account
python manage.py createsuperuser    # Django admin
python manage.py create_superuser   # App-level superadmin (superadmin/admin123)

# 6. Run the development server
python manage.py runserver

# 7. Visit http://127.0.0.1:8000/
```

8.3 Static Files for Production

Before deploying, collect static assets into staticfiles/ so WhiteNoise can serve them:

```
python manage.py collectstatic
```

9. Deployment

The project is configured to deploy on Render:

- Procfile launches the app with gunicorn.
- ALLOWED_HOSTS and CSRF_TRUSTED_ORIGINS are set to the Render domain (smart-service-booking-system.onrender.com) in settings.py.
- DEBUG is set to False; the secret key is read from the DJANGO_SECRET_KEY environment variable, with a placeholder fallback that must be overridden in production.
- WhiteNoise middleware serves collected static files directly from the app, without needing a separate static file host.

Before going live: set a real DJANGO_SECRET_KEY environment variable, change the default superadmin password, and update ALLOWED_HOSTS / CSRF_TRUSTED_ORIGINS if the domain changes.

10. Common Operating Procedures

10.1 Reset a user's password

```
python manage.py changepassword <username>
```

10.2 Promote or inspect a superadmin

Superadmin status lives on UserProfile.user_type. Use the Django shell or the create_superuser management command to set it; the superadmin_required decorator in accounts/views.py enforces access.

10.3 Adding a new page

- Public page → create a template extending base.html, add a view, wire it up in the app's urls.py.
- Dashboard page → extend dashboard_base.html instead, so it inherits the sidebar and header.

10.4 Adding new static assets

- CSS → static/css/, JS → static/js/, images → static/images/.
- Reference with {% load static %} and {% static 'path/to/file' %} in templates.

11. Known Limitations & Roadmap

11.1 Known Limitations

- The legacy OldProvider model still exists in the schema purely for historical data and should not be extended.
- SQLite is used as the database, which is fine for development/small deployments but should be swapped for PostgreSQL for heavier production traffic.

- The default superadmin password (admin123) must be changed immediately after first login on any real deployment.

11.2 Suggested Next Steps

- Email verification for new accounts.
- Payment integration for paid bookings.
- Password-reset flow via email.
- Provider ratings/reviews tied to completed bookings.
- Automated booking reminders (leveraging the existing Notification model).